Esercizio 1 (11 punti).

Sia data una CPU con processore a **5GHz** e **4 CPI** (Clock per Instruction) che adoperi indirizzi da 32 bit e memoria strutturata su due livelli di **cache** (L1, L2), il cui setup è come segue:
L1 è una cache **set-associativa** a **2 vie** con **2 set** e **blocchi da 8 word**; adopera una politica di rimpiazzo **LRU**. Ricordiamo che consideriamo la linea come l'insieme del blocco in cache con tag e bit di validità, mentre il set è il gruppo di linee con il medesimo indice.
L2 è una cache **set-associativa** a **2 vie** con **8 set** e **blocchi da 128 word** con politica di rimpiazzo **LRU**.

1) Supponendo che all'inizio nessuno dei dati sia in cache, indicare quali degli accessi in memoria indicati di seguito sono HIT o MISS in ciascuna delle due cache. Per ciascuna **MISS** indicare se sia di tipo Cold Start (**Cold**), Capacità (**Cap**) o Conflitto (**Conf**). Utilizzare la tabella sottostante per fornire i risultati. Ad esempio, scrivere **MISS** nella riga "HIT/MISS" e **Cold** nella riga "Miss type" per indicare una miss di tipo cold start.

	Address	46	66	7596	88	8800	8700	7720	1600	8336	72	1620	7612
L1	Block#												
	Index												
	Tag												
	HIT/MISS												
	Miss type												
L2	Block#												
	Index												
	Tag												
	HIT/MISS												
	Miss type												

2) Calcolare le dimensioni in bit (compresi i bit di controllo ed assumendo che ne basti uno per la LRU, e ignorando il bit dirty) delle due cache: (a) L1 e (b) L2.
3) Assumendo che gli accessi in **memoria** impieghino **800 ns**, che gli **hit** nella cache **L1** impieghino **10 ns** e gli **hit** nella cache **L2** impieghino **40 ns**, calcolare (a) il **tempo totale** per la sequenza di accessi, (b) il tempo **medio** per la sequenza di accessi, e (c) **quante istruzioni** vengono svolte nel tempo medio calcolato.
4) Calcolare il word offset del **nono** indirizzo per la cache L2 spiegando i calcoli effettuati.
5) Supponendo che gli indirizzi nella tabella siano virtuali e la memoria virtuale consti di **64 pagine** di **1 KiB** ciascuna, indicarne i numeri di pagina virtuale.

	Address	46	66	7596	88	8800	8700	7720	1600	8336	72	1620	7612
	Page#												

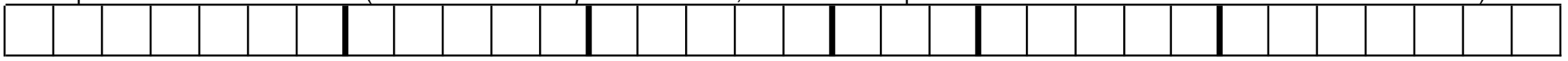
Esercizio 2 (11 punti).

Considerare l'architettura RISC-V a ciclo singolo nella figura in basso (e in allegato).
Si vuole aggiungere alla CPU l'istruzione **alujump** (`ajmp`), di **tipo R** e sintassi assembly:
`ajmp` **`res, op1, op2, switch`**
Dove res, op1, op2, e switch sono rispettivamente i campi rd, rs1, rs2, e funz3 dell'istruzione.
L'istruzione deve operare come segue:
a) Vengono letti i valori op1 e op2 dal banco registri
b) se il bit meno significativo di funz3 è 0, viene calcolata la somma op1 + op2, altrimenti viene effettuata la sottrazione op1 - op2
c) il risultato del punto b viene salvato nel registro di indice **res**
d) il PC viene aggiornato con il valore calcolato al punto b).

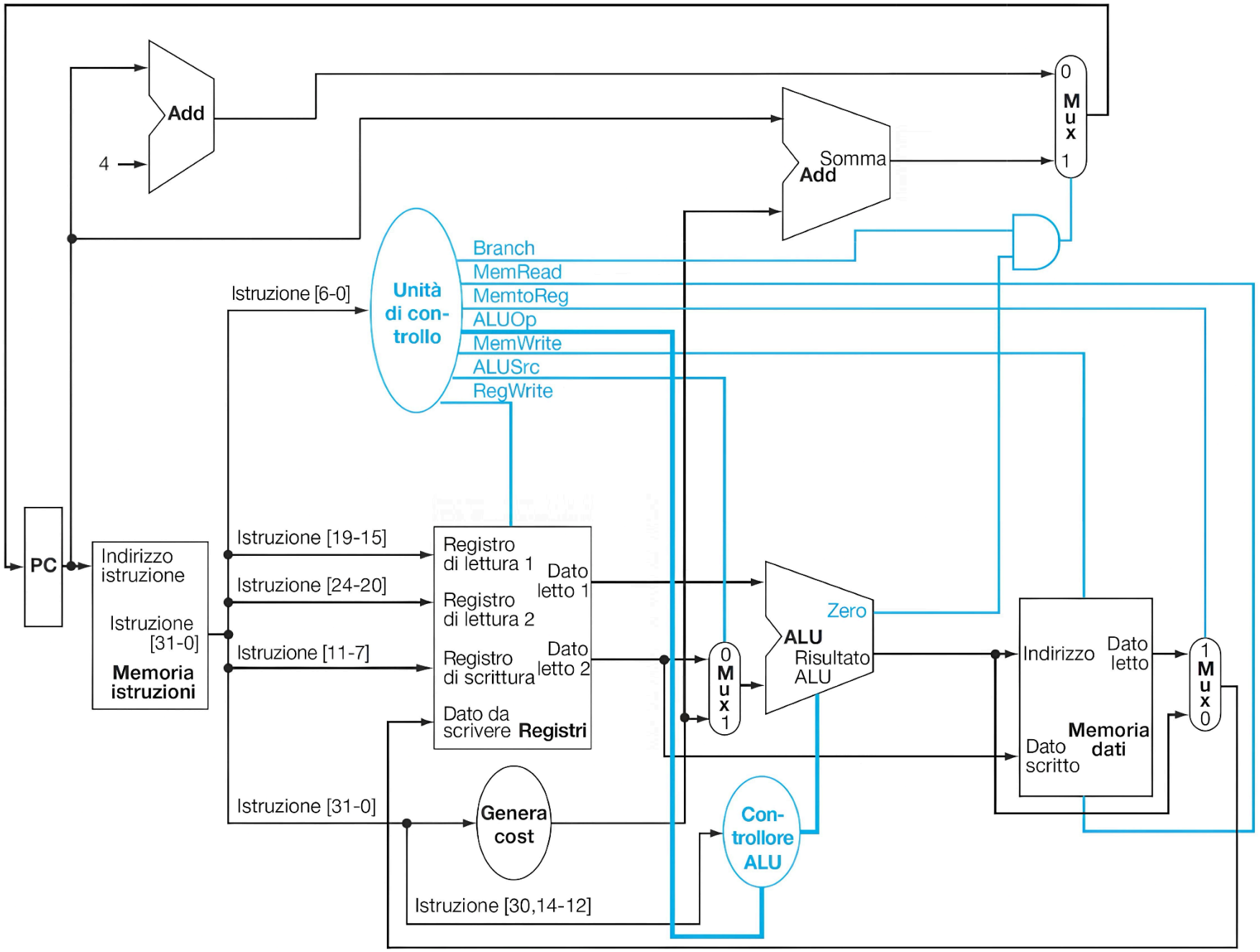
Ricordiamo che ALUOp = 00 corrisponde alla somma (0010 come controllo della ALU), ALUOp = 01 corrisponde alla sottrazione (0110 come controllo della ALU), e ALUOp = 10 corrisponde ad altre operazioni definite dal controllore della ALU usando il campo funz7.
Esempio: Supponiamo che res=1, op1=10, op2=20, switch=0b001, ovvero venga eseguita l'istruzione `ajmp 1, 10, 20, 1.`
Assumiamo che il registro x10 contenga il valore 100 e x20 il valore 50.
In tal caso:

- verranno letti i valori di x10 e x20 dal banco dei registri,
- x10 - x20 = 100 - 50 = 50 verrà salvato nel registro x1,
- Il PC verrà aggiornato con il valore 50.

1) Mostrare le **modifiche all'architettura** della CPU RISC-V, avendo cura di aggiungere eventuali altri componenti necessari a realizzare l'istruzione. A tal fine, si può alterare la stampa del diagramma architetturale oppure ridisegnare i componenti interessati dalla modifica, avendo cura di indicare i fili di collegamento e tutti i segnali entranti ed uscenti. Indicare inoltre sul diagramma i **segnali di controllo** che la CU genera *per realizzare l'istruzione*.
   Si richiede di usare solo componenti elementari (MUX, shift logici di un valore costante, operatori logici e sommatori) per realizzare questa istruzione. .
2) Indicare il contenuto in bit della word che esprime l'istruzione
   `ajmp` **`10, 0x1A, 21, 0x3`**
   compilando la tabella sottostante (assumiamo che lo *OpCode* sia `0x3A`, e che eventuali porzioni inutilizzate dall'istruzione siano codificate con zeri).



3) Supponendo che l'accesso alle **memorie** impieghi **200 ns**, l'accesso ai **registri 50 ns**, le operazioni dell'**ALU e** dei **sommatori 150 ns**, e che gli altri ritardi di propagazione dei segnali siano trascurabili, indicare la durata totale del **ciclo di clock** tale che anche l'esecuzione della nuova istruzione sia permessa *spiegando i calcoli effettuati*.
4) Descrivere nel dettaglio il funzionamento delle linee di controllo Branch e Zero nell'architettura RISC-V, e come esse influenzino l'aggiornamento del Program Counter.


Add
4
0
M
u
x
1
Somma
Add
Branch
MemRead
MemtoReg
ALUOp
MemWrite
ALUSrc
RegWrite
Unità di con-trollo
Istruzione [6-0]
PC
Indirizzo istruzione
Istruzione [31-0]
Memoria istruzioni
Istruzione [19-15]
Istruzione [24-20]
Istruzione [11-7]
Registro di lettura 1
Registro di lettura 2
Registro di scrittura
Dato da scrivere
Dato letto 1
Dato letto 2
Registri
Zero
ALU
Risultato ALU
Indirizzo
Dato letto
Dato scritto
Memoria dati
Istruzione [31-0]
Genera cost
Istruzione [30,14-12]
Con-trollore ALU

Esercizio 3 (11 punti)

Si consideri l'architettura RISC-V con pipeline. Il programma qui di seguito effettua la somma dei soli valori multipli di 4, dopo averli incrementati di 2, dall'array di lunghezza 9 formato da 3 numeri multipli di 4 e 6 numeri non multipli di 4. Infine, viene stampato il valore di tale somma. Le operazioni nelle righe 7 e 8 servono per caricare l'indirizzo base di `vettore` in `s0`, e sono equivalenti a due addi dal punto di vista della pipeline (la prima scrive su `s0` e la seconda legge da e scrive su `s0`)

```
1   .data
2   vettore: .word 24, 1, 46, 54, 12, 2, 11, 39, 4
3    # 6 non multipli di 4, 3 multipli di 4
4
5   .text                       # Sommo solo i multipli di 4
6   main:   addi t0, zero, 0    # somma parziale
7           auipc s0, 0         # operazione di tipo lui che fa s0 = PC + 0
8           addi s0, s0, 1020   # s0 ora contiene l'indirizzo base del vettore
9           addi t4, s0, 36     # primo indirizzo fuori dall'array
10  ciclo:  lw   t3, 0(s0)
11          andi t2, t3, 0x3    # è multiplo di 4?
12          bne  t2, zero, salta # se non lo è, salto
13          addi t3, t3, 2      # altrimenti aggiungo 2
14          add  t0, t0, t3     # e lo accumulo
15  salta:  addi s0, s0, 4      # prossimo elemento
16          blt  s0, t4, ciclo  # fine del ciclo?
17
18          addi a7,x0, 1       # stampa…
19          mv a0, t0           # … la somma
20          ecall
```

Si supponga che non si faccia uso di alcuna pseudoistruzione, e che le ecall non richiedano alcuno stallo. Si consideri che la decisione di branch venga presa in fase di ID, e che le unità di forwarding (quando usate) includano il forwarding da EXE a EXE, da MEM a EXE, e (per il branch) da EXE a ID.

Si indichino (ignorando hazard che possano concernere la ecall):

1) Per i primi 16 cicli di clock (**senza unità di forwarding**), le istruzioni tra le quali sono presenti **data hazard** – indicare i numeri di linea N ed M (visibili alla sinistra delle linee di codice in figura) ed il registro coinvolto xY (nel formato N / M / xY, ad esempio 17 / 18 / t0 se c'è un data hazard causato dal registro t0 tra l'istruzione alla linea 17 e quella alla linea 18);

2) Per i primi 18 cicli di clock (**con unità di forwarding**) le istruzioni tra le quali sono presenti **control hazard** – indicare i numeri di linea N ed M (nel formato N / M, ad esempio se l'istruzione alla linea 17 causa un control hazard con l'istruzione alla linea 23 scrivere: 17 / 23);

3) quanti **cicli di clock** sono necessari ad eseguire **con unità di forwarding**:

a) il **primo** loop (etichetta “ciclo”) spiegando il calcolo effettuato (indicando il tipo e numero di stalli)

b) **tutto** il programma spiegando il calcolo effettuato (usando la tabella fornita e indicando il tipo e numero di stalli)

sezione	cicli di clock richiesti	numero di occorrenze	totale

4) quanti **cicli di clock** sarebbero necessari ad eseguire **senza forwarding**:

a) il **primo** loop (etichetta “ciclo”) spiegando il calcolo effettuato (indicando il tipo e numero di stalli)

b) **tutto** il programma spiegando il calcolo effettuato (usando la tabella fornita e indicando il tipo e numero di stalli)

sezione	cicli di clock richiesti	numero di occorrenze	totale

5) quali sono, per ognuna delle cinque fasi, le **istruzioni** (o le bolle) in pipeline durante il **18° ciclo di clock** (**con forwarding**);

6) Spiegare brevemente il concetto di delay slot o salto ritardato, nel contesto della predizione dei salti.